

Einsendeaufgabe ARC-E1- Finale Architektur & ARC-E2: Architecture Communication Canvas

Projekt: Software Quality Platform

1 Einleitung

Im Rahmen dieser Einsendeaufgabe wurde für das Projekt **Software Quality Platform** eine Softwarearchitektur entworfen. Ziel ist die Entwicklung einer webbasierten Plattform zur Unterstützung der Softwarequalität. Die Plattform integriert Buildmanagement, automatisierte Unit-Tests, statische Codeanalyse sowie die Auswertung von Qualitätsmetriken in einer gemeinsamen Architektur.

Im ersten Teil der Einsendeaufgabe wird die Gesamtarchitektur anhand eines Komponentendiagramms beschrieben und erläutert. Anschließend wird für einen ausgewählten Architekturbaustein ein Architecture Communication Canvas erstellt, um dessen Aufgaben, Schnittstellen und Zusammenhänge verständlich darzustellen.

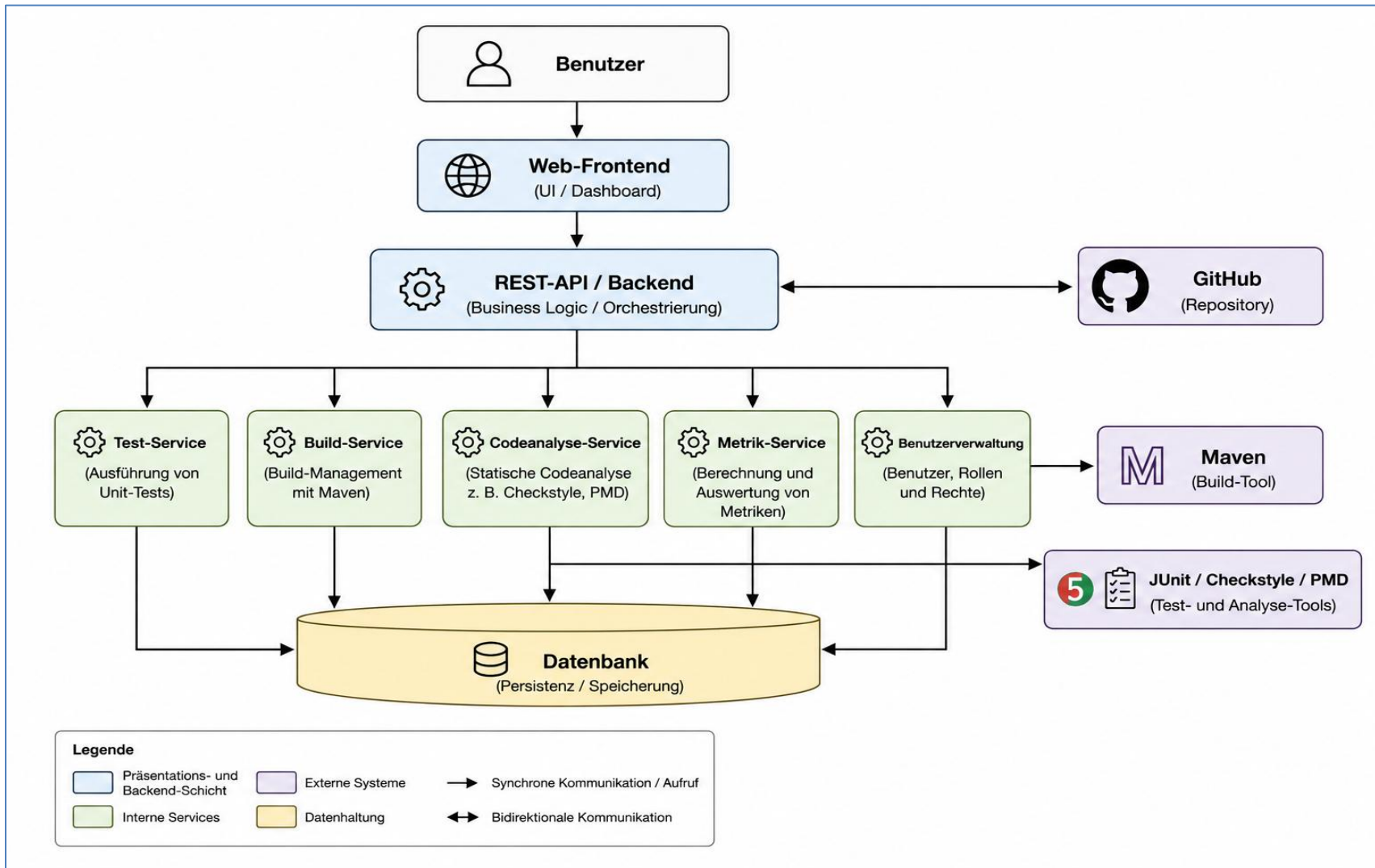


Abbildung 1: Komponentendiagramm der Software Quality Platform

2 Architekturentwurf (ARC-E1)

Abbildung 1 zeigt die Architektur der **Software Quality Platform**. Die Anwendung folgt einer mehrschichtigen Architektur und trennt Benutzeroberfläche, Geschäftslogik, interne Dienste sowie die Datenhaltung voneinander.

Der Benutzer greift über das Web-Frontend auf die Anwendung zu. Das Backend übernimmt die zentrale Steuerung der Geschäftslogik und kommuniziert mit den einzelnen Services. Diese sind für die Ausführung von Unit-Tests, das Buildmanagement, die statische Codeanalyse, die Auswertung von Qualitätsmetriken sowie die Benutzerverwaltung verantwortlich.

Externe Werkzeuge wie **Maven**, **JUnit**, **Checkstyle**, **PMD** und **GitHub** werden über definierte Schnittstellen eingebunden und unterstützen den Entwicklungs- und Qualitätssicherungsprozess.

3 Erläuterung der Architektur

3.1 Verwendete Prinzipien

Für die Architektur wurden die Prinzipien der Trennung von Verantwortlichkeiten (Separation of Concerns), der Modularisierung sowie der losen Kopplung verwendet. Jeder Service übernimmt eine klar definierte Aufgabe und kann unabhängig von den übrigen Komponenten weiterentwickelt werden.

3.2 Ebenen

Die Anwendung besteht aus einer Präsentationsschicht (Web-Frontend), einer Anwendungsschicht (REST-API und Services) sowie einer Datenschicht (Datenbank). Diese Schichten sind klar voneinander getrennt und kommunizieren über definierte Schnittstellen.

3.3 Architekturstil

Die Software Quality Platform folgt einer mehrschichtigen (Layered) Architektur mit serviceorientierten Komponenten. Dadurch bleiben die einzelnen Funktionen übersichtlich strukturiert und können unabhängig voneinander erweitert werden.

3.4 Verwendete Frameworks und Werkzeuge

Für die Entwicklung und Qualitätssicherung werden die Werkzeuge **Maven**, **JUnit**, **Checkstyle** und **PMD** eingesetzt. Maven übernimmt das Buildmanagement und die Verwaltung von Abhängigkeiten. JUnit dient zur Durchführung automatisierter Unit-Tests. Checkstyle und PMD unterstützen die statische Analyse des Quellcodes und helfen dabei, Qualitätsmängel frühzeitig zu erkennen.

3.5 Qualität und Metriken

Die Überwachung der Softwarequalität erfolgt durch automatisierte Unit-Tests sowie statische Codeanalysen. Die Ergebnisse von Checkstyle und PMD werden regelmäßig ausgewertet, um Regelverstöße und potenzielle Schwachstellen frühzeitig zu erkennen. Dadurch kann die Qualität der Software kontinuierlich verbessert werden.

4 Architecture Communication Canvas (ARC-E2)

 Architecture Communication Canvas		System	Software	created by	created for	date / iteration
		Quality Platform		Olaf Gustke	Softwaretechnik ARC-E2	27.06.2026

Value Proposition  Unterstützung der Software-qualität durch Tests, Codeanalyse und Qualitätsmetriken.	Core Functions  * Buildmanagement * Unit-Tests * Codeanalyse * Metriken * Benutzerverwaltung	Core Decisions - Good or Bad  Layered Architecture, REST-API, modulare Services und zentrale Datenhaltung.	Technologies  Java, Maven, JUnit, Checkstyle, PMD und GitHub.
Key Stakeholder  Entwickler, Projektleitung und Qualitätssicherung.	Quality Requirements  Wartbarkeit, Testbarkeit, Erweiterbarkeit und hohe Codequalität.		
Business Context  Unterstützung von Entwicklern bei Build, Test, Codeanalyse und Qualitätsüberwachung.		Components / Modules  Web-Frontend, Backend, Test-Service, Build-Service, Codeanalyse-Service, Metrik-Service, Benutzerverwaltung und Datenbank.	
Risks and Missing Information  Fehlkonfiguration, fehlerhafte Builds oder unvollständige Codeanalysen.			


 Architecture Communication Canvas, © 2023 by Gernot Starke, Patrick Roos, Benjamin Wolf and arc42 Contributors is licensed under Attribution-ShareAlike 4.0 International.

Abbildung 2: Architecture Communication Canvas der Software Quality Platform

Abbildung 2 zeigt den Architecture Communication Canvas der Software Quality Platform. Er fasst den Nutzen, die Kernfunktionen, die verwendeten Technologien, die Qualitätsanforderungen sowie die wichtigsten Architekturentscheidungen kompakt zusammen und ergänzt damit das Komponentendiagramm aus ARC-E1.

5 Fazit

Im Rahmen dieser Einsendeaufgabe wurde eine Architektur für die **Software Quality Platform** entwickelt und anhand eines Komponentendiagramms visualisiert. Die gewählte Architektur trennt die einzelnen Verantwortungsbereiche klar voneinander und unterstützt eine gute Wartbarkeit sowie Erweiterbarkeit der Anwendung. Mithilfe des Architecture Communication Canvas wurde zusätzlich ein Architekturbaustein detaillierter beschrieben. Dadurch konnten sowohl die Gesamtarchitektur als auch die Kommunikation eines einzelnen Bausteins nachvollziehbar dokumentiert werden.

Die gewählte Architektur ermöglicht eine klare Trennung der Verantwortlichkeiten und unterstützt dadurch eine nachhaltige Weiterentwicklung der Software.

6 Quellen

<https://canvas.arc42.org>

<https://maven.apache.org>

<https://junit.org/junit5>

<https://checkstyle.sourceforge.io>

<https://pmd.github.io>

<https://arc42.org>